

RELATÓRIO DE TESTES

Cart Checkout + Mock Payment System

André Carvalho Bonifacio, Francine de Oliveira Delanni e Lucas Melo Vinhatti

1. Introdução

O presente relatório trata-se de um relato sobre os testes desenvolvidos pela equipe para o Cart Checkout + Mock Payment System, com o intuito de avaliar e ampliar a cobertura de testes de uma aplicação de e-commerce que simula o fluxo completo de uma compra online. O esforço aqui descrito buscou não apenas verificar o comportamento do sistema, mas também identificar lacunas na suíte de testes original e propor casos que cobrissem as regras de negócio mais sensíveis da aplicação, como a idempotência de operações e a retentativa de pagamento após falha.

O sistema sob análise possui as funcionalidades de criação de carrinho, a adição de itens com validação de estoque, o checkout, o início de pagamento e o processamento de webhooks de um provedor de pagamento simulado. Além das operações usuais de CRUD, foram identificadas as seguintes regras de negócio como centrais para o funcionamento correto da aplicação:

- máquinas de estados do Pedido (Order: CREATED → PENDING_PAYMENT → PAID / PAYMENT_FAILED) e do Pagamento (Payment: PENDING → CONFIRMED / FAILED);
- idempotência de checkout, de início de pagamento e de webhooks duplicados;
- retentativa (retry) de pagamento após falha, com criação de um novo registro de Payment que preserva o histórico do pagamento que falhou;
- validação de estoque tanto na composição do carrinho quanto na finalização do pedido.

A aplicação segue uma arquitetura em camadas (controller, application, domain e repository), sendo as regras de transição de estado mantidas explicitamente no modelo de domínio, sem uso de bibliotecas de máquina de estados externas. Essa escolha arquitetural concentra grande parte da lógica de negócio em um ponto passível de testes unitários isolados.

1.1 Código-fonte

O código-fonte do projeto, que está disponível publicamente no repositório do GitHub, e o vídeo com os testes sendo executados, estão nos endereços a seguir.

Repositório: <https://github.com/A-Samyy/cart-checkout-payment>

Vídeo: <https://www.youtube.com/watch?v=7sAyP8sIPAQ>

Item	Detalhe
Linguagem	Java 21
Framework principal	Spring Boot 4.0.6
Build	Maven (com Maven Wrapper - ./mvnw)
Banco de dados (testes)	H2 em memória (profile test)

1.2 Frameworks e APIs de teste utilizados

Para a construção da suíte de testes, nós optamos pelas ferramentas já consolidadas no ecossistema Java/Spring, de modo a manter consistência com o que já vinha sendo utilizado no projeto original. As ferramentas empregadas e o respectivo uso estão descritos na tabela a seguir.

Ferramenta	Uso
JUnit 5	Framework de testes (@Test, @Nested, @DisplayName)
AssertJ	Asserções fluentes (assertThat, assertThatThrownBy)
Mockito	Testes unitários da camada de aplicação (@ExtendWith(MockitoExtension.class))
Spring Boot Test	Testes de integração (@DataJpaTest) e de sistema (@SpringBootTest)
MockMvc	Simulação de requisições HTTP nos testes de sistema
H2	Banco em memória para testes de integração e de sistema

1.3 Como compilar e rodar os testes

Como pré-requisito, é necessário possuir o Java 21 instalado. O projeto inclui o Maven Wrapper (./mvnw), o que dispensa a instalação global do Maven na máquina de quem for reproduzir os testes. Os comandos utilizados para compilar a aplicação e executar a suíte de testes, de forma completa ou pontual, estão descritos a seguir.

- Compilar e executar a aplicação: mvn spring-boot:run
- Rodar toda a suíte de testes: ./mvnw clean test
- Rodar uma classe específica: ./mvnw test -Dtest=PaymentServiceTest
- Rodar um único método de teste:

```
./mvnw test -Dtest=PaymentRetryHistorySystemTest#retryPreservesFailedPaymentHistoryAndIgnoresLateWebhook
```

Cabe uma observação importante sobre a versão do Spring Boot utilizada: a partir da versão 4, o slice @DataJpaTest deixou de vir incluído no spring-boot-starter-test, passando a exigir a dependência spring-boot-starter-data-jpa-test (escopo test), que foi adicionada ao pom.xml. Os imports também sofreram alteração de pacote, passando a ser:

- org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest
- org.springframework.boot.jpa.test.autoconfigure.TestEntityManager
- org.springframework.boot.jdbc.test.autoconfigure.AutoConfigureTestDatabase.

Sem essa atualização, os testes de integração desta etapa não seriam sequer compilados.

2. Análise

Antes da implementação dos casos de teste desta etapa, nós realizamos uma análise crítica da qualidade da suíte já existente no repositório original, com o objetivo de identificar o que já estava bem coberto e onde havia lacunas. Essa análise serviu de base para a definição dos grupos de teste apresentados na seção 4.

2.1 Testes presentes no repositório original

O repositório continha, antes da presente etapa, quatro classes de teste, situadas em diferentes níveis da pirâmide de testes, conforme mostra a tabela a seguir.

Teste	Tipo	Nível na pirâmide
CartDomainTest	Unitário (domínio puro, sem Spring)	Unidade
OrderStateMachineTest	Unitário (máquina de estados)	Unidade
PaymentDomainTest	Unitário (máquina de estados)	Unidade
CheckoutPaymentFlowIntegrationTest	@SpringBootTest + MockMvc	Sistema / e2e

2.2 Pontos fortes

Observou-se que os testes de domínio já existentes eram unitários "puros": rápidos, sem contexto Spring, e focados em invariantes de negócio, como as transições de estado válidas e inválidas do pedido e do pagamento. Constatou-se também que o fluxo principal da aplicação já estava coberto de ponta a ponta, incluindo um cenário de retentativa de pagamento (paymentFailureThenRetry), o que indicava que a base da suíte estava bem direcionada, ainda que incompleta.

2.3 Fraquezas identificadas

Apesar dos pontos fortes citados, a análise revelou cinco fraquezas relevantes na suíte original, descritas a seguir.

A primeira delas foi o que nós chamamos de "pirâmide oca no meio": existiam testes na base (unidade de domínio) e no topo (sistema), mas nada testava a application layer (PaymentService) de forma isolada, tampouco a repository layer. Isso significa que as decisões mais críticas do sistema, como PaymentService.handleWebhook e PaymentService.startPayment, só eram exercitadas indiretamente, através do teste de sistema.

A segunda fraqueza está relacionada a uma classificação enganosa do tipo de teste: o CheckoutPaymentFlowIntegrationTest, apesar do nome, sobe o contexto Spring inteiro, sendo na prática um teste de sistema e não um teste de integração. Some-se a isso o uso de @DirtiesContext em todos os métodos, o que recria o contexto a cada teste e penaliza consideravelmente o tempo total de execução da suíte.

A terceira lacuna identificada foi a cobertura incompleta da matriz de decisão do webhook em nível unitário. A lógica de tratamento do webhook corresponde a uma matriz 3x2 (status PENDING/CONFIRMED/FAILED cruzado com resultado CONFIRMED/FAILED), e, em nível de service isolado, faltavam as combinações "FAILED+FAILED" e "FAILED+CONFIRMED".

A quarta fraqueza diz respeito à ausência de um valor-limite exato na fronteira de estoque: o CartDomainTest validava estoque insuficiente com uma quantidade genérica (10 em estoque de 5), mas não exercitava a borda exata, ou seja, quantidade igual ao estoque e quantidade igual a estoque mais um.

Por fim, constatou-se que o teste de retry existente (paymentFailureThenRetry) validava apenas que o segundo pagamento era diferente do primeiro, sem consultar o pagamento falho via API nem testar o recebimento de um callback tardio vindo do provedor.

2.4 Observações técnicas

Verificou-se que o projeto já possuía o arquivo `src/test/resources/application-test.properties` corretamente configurado, com `H2` em memória, `ddl-auto=create-drop` e inicialização via SQL, fazendo com que `@ActiveProfiles("test")` funcionasse como esperado. Notou-se ainda que o uso de `@DirtiesContext` nos testes de sistema concentra grande parte do tempo de execução da suíte: as duas classes de sistema, `CheckoutPaymentFlowIntegrationTest` (aproximadamente 5s) e `PaymentRetryHistorySystemTest` (~5s), respondem por cerca de 10 dos ~15 segundos totais medidos com `./mvnw clean test`.

3. Objetivos

Esta seção descreve o objetivo dos testes implementados nesta etapa, a partir da definição de uma jornada de usuário representativa do domínio, seguindo as técnicas usuais de Engenharia de Software para especificação de jornadas: persona, objetivo, pré-condições, fluxo principal, fluxo alternativo e pós-condições.

3.1 Jornada de usuário selecionada

Para direcionar a construção dos casos de teste, optamos por partir de uma jornada específica, e não de uma cobertura genérica de CRUD, por entender que essa abordagem cruza as regras de negócio mais sensíveis do sistema. A jornada escolhida está descrita na tabela abaixo.

Elemento	Descrição
Persona	Cliente que finalizou um carrinho e vai efetuar o pagamento
Objetivo	Pagar um pedido cuja primeira tentativa de pagamento falhou
Pré-condições	Existe um Order em estado CREATED (checkout já concluído)
Fluxo principal	Inicia pagamento → provedor responde FAILED → cliente reinicia o pagamento (retry) → provedor responde CONFIRMED → pedido fica PAID
Pós-condições	Pedido PAID; pagamento confirmado ativo; pagamento falho preservado no histórico
Fluxo alternativo	Webhook tardio sobre o pagamento antigo (já FAILED) é ignorado sem corromper o pedido PAID

As regras de negócio atravessadas por essa jornada envolvem a máquina de estados de Order e de Payment, a idempotência do webhook, a criação de um novo Payment no retry, preservando o histórico do que falhou, e a distinção, na persistência, entre pagamento PENDING ativo e FAILED antigo. Considerou-se que essa jornada não se resume a um CRUD, sendo, portanto, a mais adequada para validar a robustez da implementação.

3.2 Objetivo dos testes implementados

Definida a jornada, o objetivo passou a ser preencher as lacunas identificadas na análise (seção 2) com testes nos três níveis exigidos, cobrindo a jornada de ponta a ponta, conforme sintetizado a seguir.

Nível	Classe	O que valida na jornada
Unitário (service)	PaymentServiceTest	Orquestração de startPayment e handleWebhook
Unitário (domínio)	CartDomainTest (C4/C5)	Fronteira exata de estoque (pré-requisito do checkout)
Integração (persistência)	PaymentRepositoryIntegrationTest	Query que distingue PENDING ativo de FAILED antigo
Sistema (e2e)	PaymentRetryHistorySystemTest	Histórico preservado + callback tardio via API

4. Casos de teste

Para o desenho dos casos de teste, foram aplicadas as técnicas de partição de equivalência, tabela de decisão e análise de valor-limite, todas técnicas de caixa-preta amplamente utilizadas em Engenharia de Software. Os testes de service, especificamente, foram implementados de forma isolada com Mockito, o que caracteriza uma abordagem de caixa-branca no nível de orquestração.

Grupo A - PaymentService.startPayment (unitário com Mockito)

Neste grupo, a técnica utilizada foi a partição de equivalência por estado do pedido e presença, ou não, de pagamento pendente. Os casos estão implementados no arquivo PaymentServiceTest.java e resumidos na tabela a seguir.

ID	Estado do Order	Payment PENDING?	Saída esperada	Verificações
A1	CREATED	Não	Retorna Payment PENDING; order vira PENDING_PAYMENT	save() 1x; captor valida status e valor
A2	PENDING_PAYMENT	Sim	Retorna o mesmo pagamento existente (idempotência)	save() nunca chamado; mesmo paymentId
A3	PAYMENT_FAILED	Não (antigo FAILED)	Retorna novo Payment PENDING (retry)	save() 1x; captor valida status e valor
A4	PAID	Não	Lança IllegalStateException	save() nunca chamado
A5	(order inexistente)	—	Lança NotFoundException	save() nunca chamado

Grupo B - PaymentService.handleWebhook (unitário com Mockito)

Neste grupo, a técnica utilizada foi a tabela de decisão, buscando cobrir a matriz completa 3×2 de status atual cruzado com resultado recebido. Assim como o grupo anterior, os casos estão implementados no arquivo PaymentServiceTest.java.

ID	Status atual	Resultado	Payment final	Order final	Save?
B1	PENDING	CONFIRMED	CONFIRMED	PAID	Sim
B2	PENDING	FAILED	FAILED	PAYMENT_FAILED	Sim
B3	CONFIRMED	CONFIRMED	CONFIRMED	PAID	Não (duplicado ignorado)
B4	CONFIRMED	FAILED	CONFIRMED	PAID	Não (já finalizado)
B5	FAILED	FAILED	FAILED	PAYMENT_FAILED	Não (duplicado ignorado)
B6	FAILED	CONFIRMED	FAILED	PAYMENT_FAILED	Não (já finalizado)
B7	(payment inexistente)	qualquer	—	NotFoundException	Não

Vale destacar que as combinações B5 e B6 eram justamente as lacunas apontadas na análise da seção 2.3, referentes a duplicidade de webhook em pagamentos já falhos, e foram implementadas nesta etapa.

Grupo C - Reforço de domínio no CartDomainTest (valor-limite)

Este grupo aplicou a técnica de análise de valor-limite sobre a quantidade do carrinho, complementando os testes já existentes no arquivo CartDomainTest.java.

ID	Operação	Estoque	Quantidade	Resultado esperado	Situação
C1	addItem	100	-1	BusinessException ("negative")	Já existia
C2	updateItemQuantity	100	0	Item removido do carrinho	Já existia
C3	addItem	10	1	Adiciona (mínimo válido)	Já existia (indireto)
C4	addItem	10	10 (== estoque)	Adiciona (limite superior válido)	Implementado nesta etapa
C5	addItem	10	11 (estoque+1)	BusinessException ("insufficient")	Implementado nesta etapa

O caso C4 exercita a borda clássica entre os operadores ">" e ">=", enquanto o caso C5 substitui o valor genérico utilizado anteriormente pelo limite exato de estoque mais um, tornando a verificação de fronteira mais precisa.

Grupo D - PaymentRepository (integração com @DataJpaTest)

Este grupo corresponde ao primeiro teste de integração real da camada de persistência contra o banco H2, implementado no arquivo PaymentRepositoryIntegrationTest.java, preenchendo a lacuna de "pirâmide oca" apontada na análise.

ID	Cenário persistido (mesmo Order)	Consulta	Saída esperada
D1	1 pagamento FAILED + 1 PENDING	findFirstByOrderIdAndStatus(PENDING)	Retorna o PENDING; FAILED continua consultável; count()==2
D2	apenas 1 pagamento FAILED	findFirstByOrderIdAndStatus(PENDING)	Optional.empty
D3	1 pagamento PENDING	findFirstByOrderIdAndStatus(PENDING)	@EntityGraph carrega a associação order (id/estado corretos)

Grupo E - Jornada de retry com histórico (sistema com MockMvc)

Por fim, este grupo corresponde ao teste de sistema (e2e) complementar ao CheckoutPaymentFlowIntegrationTest, implementado no arquivo PaymentRetryHistorySystemTest.java, e é o que efetivamente exercita a jornada de usuário completa definida na seção 3.1.

ID	Passos	Verificações
E1	checkout → start (p1) → webhook FAILED → start (p2 ≠ p1) → webhook CONFIRMED → webhook tardio FAILED em p1	GET /payments/p1 = FAILED; GET /payments/p2 = CONFIRMED; webhook tardio → "Duplicate webhook ignored safely"; pedido permanece PAID

Mapa de cobertura vs. pirâmide de testes

De modo a consolidar a relação entre os grupos de teste descritos e os níveis da pirâmide de testes, foi elaborado o mapa a seguir.

Nível	Classe	Grupos	Status
Unitário - service (Mockito)	PaymentServiceTest	A, B	Nova (12 testes)
Unitário - domínio (valor-limite)	CartDomainTest	C	Alterada (+2 testes)
Integração - persistência (@DataJpaTest)	PaymentRepositoryIntegrationTest	D	Nova (3 testes)
Sistema - e2e retry + histórico	PaymentRetryHistorySystemTest	E	Nova (1 teste)
Sistema - e2e (regressão)	CheckoutPaymentFlowIntegrationTest	—	Mantida (9 testes)

Com os Grupos A a E, a pirâmide de testes passa a cobrir a jornada de retry em nível de unidade de service, integração de persistência e sistema, complementando a base de domínio que já existia previamente.

5. Resultado dos testes

A execução completa da suíte foi realizada via `./mvnw clean test`, em ambiente Java 21, Maven 3.9.15, com H2 em memória no profile test. Todos os testes passaram: foram 48 testes, 0 falhas e 0 erros, em um tempo total de aproximadamente 15 segundos, conforme detalhado na tabela a seguir.

Classe de teste	Testes	Falhas	Erros	Tempo aprox.
PaymentServiceTest (nova)	12	0	0	0,5s
PaymentRepositoryIntegrationTest (nova)	3	0	0	0,7s
PaymentRetryHistorySystemTest (nova)	1	0	0	5s
CartDomainTest (+2 novos)	9	0	0	0,02s
OrderStateMachineTest	8	0	0	0,1s
PaymentDomainTest	6	0	0	0,01s
CheckoutPaymentFlowIntegrationTest	9	0	0	5s
Total	48	0	0	15s

Anexo A - Resumo das alterações no código

Arquivo	Alteração
payment/application/PaymentServiceTest.java	Nova classe - 12 testes (Grupos A e B)
payment/repository/PaymentRepositoryIntegrationTest.java	Nova classe - 3 testes (Grupo D)
PaymentRetryHistorySystemTest.java	Nova classe - 1 teste (Grupo E)
cart/domain/CartDomainTest.java	adição de 2 testes (C4 e C5)
pom.xml	adição do spring-boot-starter-data-jpa-test (escopo test)

Anexo B - Uso da inteligência artificial

PROMPT 1:

Anexamos a descrição do trabalho e usamos o seguinte prompt:

“Me ajude a entender sobre do que se trata o trabalho, me dê um descrição em detalhes de como eu posso fazer o que está sendo pedido, e como eu posso dividir as tarefas entre uma equipe de 3 pessoas, divida em etapas/roteiro de desenvolvimento”